CHEATSHEET RINGKAS C++ — PROBLEM 4

Cara jawab panitia

Jelaskan urutannya: (1) masalahnya apa, (2) ide algoritmanya, (3) fungsi/variabel yang menjalankan ide itu, (4) kompleksitasnya. Jangan mengaku paham detail yang belum kamu pahami; jelaskan bagian yang kamu tahu dengan jujur.

C++ kilat

cin >> x membaca input; cout << x mencetak output; for mengulang; vector<T> mirip list Python; long long integer 64-bit untuk angka besar; sort mengurutkan; __gcd(a,b) mencari FPB; % k mengambil sisa bagi. Array C++ pada beberapa program ini dimulai dari indeks 1, jadi indeks 0 atau n+1 kadang dipakai sebagai batas bantu.

Angka besar yang kelihatan random

100005 = kira-kira batas 100.000 elemen + ruang indeks tambahan. 1005 = kira-kira batas 1.000 baris/kolom + ruang tambahan. 200005 dan 300005 = batas sekitar 200.000 dan 300.000 + ruang tambahan. 800005 = sekitar 4 × 200.000 untuk kapasitas segment tree. 1000000000 = nilai sentinel awal yang sengaja sangat besar; nanti diganti nilai nyata. Ukuran final seharusnya mengikuti batas maksimum di soal.

Kompleksitas

O(n) artinya waktu tumbuh sebanding jumlah data; O(n log n) biasanya karena sorting atau sparse table; O(nm) berarti memproses tiap sel grid; O(log n) berarti membagi pencarian lewat struktur pohon. Kompleksitas adalah perkiraan pertumbuhan waktu, bukan detik pasti.

1. BATU BERTUAH

Tujuan kode: mencoba membuang k elemen berurutan dan memilih posisi buang yang membuat FPB (GCD) elemen tersisa sebesar mungkin.

Alur

```
1) Baca n, k, dan array a. 2) pre[i] = FPB a[1..i]. 3) suf[i] = FPB a[i..n]. 4) Jika blok yang dibuang mulai di i, FPB sisa = gcd(pre[i-1], suf[i+k]). 5) Simpan nilai terbesar dan posisi awalnya.
```

Kode inti

```
pre[i] = __gcd(pre[i-1], a[i]);
suf[i] = __gcd(suf[i+1], a[i]);
int g = __gcd(pre[i-1], suf[i+k]);
if (g > gcdBest) { gcdBest = g; idxBest = i; }
```

Fungsi/angka

pre = FPB dari kiri; suf = FPB dari kanan; __gcd = fungsi FPB. pre[0]=0 dan suf[n+1]=0 dipakai sebagai batas; FPB(0,x)=x. 100005 menyediakan kapasitas sekitar 100 ribu elemen dengan indeks 1-based.

Tanya panitia

Q: Kenapa tidak menghitung FPB sisa dari awal untuk tiap posisi? A: Itu mengulang banyak kerja; prefix dan suffix membuat FPB bagian kiri/kanan siap dipakai. Q: Kenapa kondisi g > gcdBest, bukan >=? A: Agar posisi pertama yang mencapai nilai terbaik tetap tersimpan. Kompleksitas: O(n log A), kira-kira linear jumlah elemen dengan biaya operasi GCD.

2. PERMAINAN BOM SILANG

Tujuan kode: untuk setiap sel (i,j), menghitung hasil kali semua angka yang tidak berada di baris i dan tidak berada di kolom j, modulo k.

Alur

1) Simpan angka A dan reduksi tiap angka dengan % k. 2) Untuk setiap baris, prefix kiri dan suffix kanan menghitung hasil kali semua elemen kecuali kolom j; simpan di E[i][j]. 3) Untuk setiap kolom j, prefix atas dan suffix bawah mengalikan E dari semua baris kecuali baris i; simpan ke hasil[i][j].

Kode inti

```
kiri[j] = kiri[j-1] * A[i][j] % k;
kanan[j] = kanan[j+1] * A[i][j] % k;
E[i][j] = kiri[j-1] * kanan[j+1] % k;
hasil[i][j] = atas[i-1] * bawah[i+1] % k;
```

Fungsi/angka

A = matriks input; E = hasil kali baris tanpa kolom j; hasil = jawaban tiap sel. 1005 cocok untuk dimensi sekitar 1000 dan indeks tambahan. 1 % k adalah nilai identitas perkalian dalam aritmetika modulo (kalau k valid dan >0). long long dipakai karena perkalian sisa modulo bisa besar.

Tanya panitia

Q: Kenapa tidak membagi total hasil kali dengan A[i][j]? A: Pembagian modulo tidak selalu valid, terutama jika k bukan prima atau ada angka yang tidak punya invers modulo. Prefix/suffix menghindari pembagian. Kompleksitas: O(nm), karena tiap sel diproses beberapa kali.

3. JUTSU SUPER

Tujuan kode: memeriksa palindrom ganjil pada setiap garis horizontal, vertikal, dan dua arah diagonal; lalu mencari sel dengan nilai minimum-arah terbesar.

Alur

jalan() mengumpulkan koordinat satu garis sesuai arah (di,dj). proses() mengubah karakter pada garis menjadi string, lalu algoritma Manacher menghitung radius palindrom di tiap pusat. Panjang palindrom ganjil = 2*d[i]-1. Nilai kuat pada tiap sel diambil minimum dari garis-garis yang melewatinya. Terakhir, hitung sel yang memiliki kuat maksimum.

Kode inti

```
void jalan(int si, int sj, int di, int dj)  // kumpulkan satu garis
void proses(vector<pair<int,int> >& garis)       // Manacher
int panjang = 2 * d[i] - 1;
kuat[x][y] = min(kuat[x][y], panjang);
```

Fungsi/angka

grid = karakter grid; garis = koordinat sel dalam satu garis; d[i] = radius palindrom Manacher; l,r = palindrom paling kanan yang dipakai ulang agar efisien. 1005 = ruang untuk ukuran grid sekitar 1000. 1000000000 di kuat adalah sentinel awal yang sengaja lebih besar daripada panjang palindrom yang mungkin; min() akan menggantinya.

Tanya panitia

Q: Apa palindrom? A: Urutan karakter yang sama jika dibaca dari dua arah. Q: Kenapa Manacher? A: Menghitung radius palindrom untuk semua pusat dalam waktu linear per garis, bukan memeriksa semua panjang berulang kali. Kompleksitas total kira-kira O(nm), karena setiap arah memproses total sel grid secara linear.

4. MEDAN ELMAX

Tujuan kode: menerima penambahan shield aktif di rentang kota, lalu tiap serangan pada kota mencari shield aktif terbaru yang masih punya HP.

Alur

Operasi tipe 1: buat shield baru dengan HP h, aktif di [a,b], dan kedaluwarsa pada t+d-1; pasang ID shield ke segment tree. Operasi lain: cari ID shield terbaru yang valid di kota a. Jika tidak ada, cetak EVAKUASI SEKARANG!; jika ada, kurangi HP dengan k dan cetak pesan shield.

Fungsi/variabel

pasang() menyimpan ID shield pada node segment tree yang mencakup rentang. cari(a,waktu) menelusuri jalur ke kota a, membuang shield paling belakang jika HP habis atau sudah kedaluwarsa, dan memilih ID terbesar/terbaru. hp[id] = HP; habis[id] = waktu akhir aktif; jumlahShield = ID yang terus bertambah.

Medan Elmax — angka/kompleksitas (lanjutan)

tree[800005] kira-kira 4 × 200.000 node, ukuran lazim untuk segment tree pada n sekitar 200.000. hp dan habis 200005 menyimpan data shield. t+d-1 membuat durasi d mencakup waktu mulai t. long long digunakan untuk HP/durasi yang mungkin besar. Update/query kira-kira O(log n), dengan pembuangan tiap ID shield paling banyak sekali secara amortisasi.

Tanya panitia

Q: Mengapa memilih ID terbesar? A: ID diberi berurutan saat shield dibuat, jadi ID lebih besar berarti lebih baru. Q: Kapan shield tidak valid? A: Saat HP <= 0 atau habis[id] < waktu saat ini.

5. SPACE-TIME REVERSAL

Tujuan kode: membandingkan skor normal—pilih hadiah terbesar di tiap ruangan—dengan skor reversal—pilih n hadiah terbesar dari semua ruangan.

Alur dan kode inti

Saat membaca tiap ruangan, maks adalah hadiah terbesar di ruangan itu, lalu ditambahkan ke normal. Semua hadiah disimpan dalam vector. Setelah diurut menurun, jumlahkan n nilai pertama untuk rev.

```
normal += maks;
sort(semua.begin(), semua.end(), greater<long long>());
for (int i=0; i<n; i++) rev += semua[i];
```

Fungsi/angka

vector<long long> semua menyimpan semua hadiah secara dinamis. long long dipakai untuk total besar. greater<long long>() membuat urutan menurun. Tidak ada konstanta array besar karena vector mengatur ukuran sendiri.

Tanya panitia

Q: Apa beda normal dan rev? A: Normal harus mengambil satu pilihan terbaik per ruangan; rev mengurutkan semua hadiah dan mengambil n hadiah terbesar tanpa batas satu per ruangan. Kompleksitas: O(nm log(nm)) karena sorting semua hadiah.

6. TEMAN ASRAMA

Ini algoritma paling rumit di antara enam file. Fokus pahami tujuan setiap blok, bukan hafalkan tiap baris.

Ide dan alur

1) Urutkan interval (L,R) berdasarkan L. 2) Gabungkan interval menjadi komponen selama interval berikutnya masih bersinggungan dengan rentang yang sedang terbentuk (kode memulai komponen baru bila L > maxR). 3) Sparse table menjawab nilai R minimum pada rentang indeks dengan cepat. 4) Prefix m1/m2/idx1 melacak R terbesar, R terbesar kedua, dan indeks pemilik R terbesar. 5) potong mencatat titik yang bisa memisahkan komponen saat satu interval dibuang. 6) Coba membuang tiap interval, hitung kandidat hasil, ambil jawaban terbesar.

Variabel penting

a[i]=(L,R) interval yang telah diurutkan. sp[k][i] = nilai minimum R untuk rentang dengan panjang 2^k. minR(l,r) = minimum R pada indeks l..r. lg[len] = floor(log2(len)). kompDari/kompAwal/kompAkhir menandai komponen. nilaiKomp = nilai sebuah komponen. m1/m2/idx1 mendukung deteksi titik potong. potong[x] berisi posisi potong yang terkait dengan interval x.

Angka dan kompleksitas

300005 = kapasitas sekitar 300.000 interval + indeks tambahan. sp[20] menyediakan level sparse table cukup untuk rentang sampai sekitar 300.000 (karena log2(300.000) < 19). long long menyimpan nilai/jawaban agar aman dari overflow. Sorting + membangun sparse table membutuhkan O(n log n); query minR setelahnya O(1). Total ditargetkan O(n log n).

Tanya panitia

Q: Apa gunanya sparse table? A: Menjawab minimum R pada rentang yang sama berulang kali dalam O(1) setelah preprocessing. Q: Kenapa harus sort? A: Supaya interval yang bertumpang tindih/bersambung bisa diproses berurutan berdasarkan L. Kalau ditanya detail rumus cand, jelaskan bahwa kode membandingkan komponen yang tidak terpengaruh dan bagian komponen yang tersisa setelah satu interval dihapus.

CATATAN WAJIB SEBELUM PRESENTASI

Nama variabel sesuai instruksi soal

Pada soal Batu Bertuah, instruksi menyebut nama variabel hasil Result__, tetapi kode mencetak gcdBest. Pada soal Teman Asrama, instruksi meminta jumlah teman disimpan di ovmPisiN, sementara kode memakai n. Jika nama variabel dicek secara literal, ini bisa menjadi masalah kepatuhan instruksi. Saat ditanya, jelaskan kerja yang dihemat algoritma; jika lupa sintaks, jelaskan ide utamanya lalu tunjuk baris kode terkait.